COMP 511

Computer Programming Foundations II

Heap

Algorithmic Complexity

- **Why don’t constants matter in algorithm complexity theory?**
  - When using **Big O notation**, our interest is in *asymptotic* behavior.
  - Since the constant is independent of n, it does not affect how the algorithm behaves as the input size grows towards infinity.
  - Big O notation is not an exact measurement. Instead, it's a way to categorize the complexity of an algorithm.

Algorithmic Complexity

	Insert	Search	Find min	Delete min
Unsorted Array	O(1) Data element may be inserted at end of array	O(n)	O(n)	O(n) Find min: O(n) Delete and fill gap: O(n) O(n) + O(n) = 2*O(n) = O(n)
Sorted Array	O(log n) to find the insertion location + O(n) to make room within the array in order to perform the insertion. O(log n) + O(n) = O(n)	O(log n)	O(1)	O(1) to find min. O(n) to re-pack the array. O(1) + O(n) = O(N)
Unsorted Linked List	O(1) Data element may be inserted at start of list.	O(n)	O(n)	O(n) to find min O(1) to delete min O(n+1) = O(n)

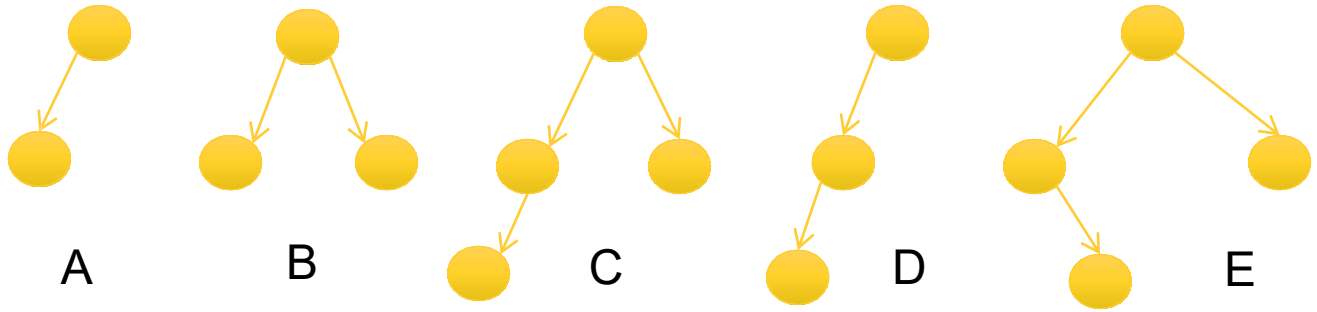
Introducing the Binary Heap

	Insert	Search	Find min	Delete min
Unsorted Array	O(1)	O(n)	O(n)	O(n)
Sorted Array	O(log n) + O(n) = O(n)	O(log n)	O(1)	O(1) + O(n) = O(N)
Unsorted Linked List	O(1)	O(n)	O(n)	O(n+1) = O(n)
Binary (Min) Heap	O(log n)		O(1)	O(log n)

Data structures are selected based on the operations predominately performed. If our application requires Insert(), FindMin(), and DeleteMin() be optimized, we will select a data structure that provides optimal performance in these areas. Notice in the above chart, that Binary Heap does not require O(n) time for any of these 3 operations.

We will be discussing two types of Heap structures: Min Heap, and Max Heap. In this class, we'll be discussing the binary tree visualization of the heap.

Recall: A **complete binary tree** is a binary tree which is completely filled, with the possible exception of the bottom level, which is filled from left to right. The heap utilizes a complete binary tree.



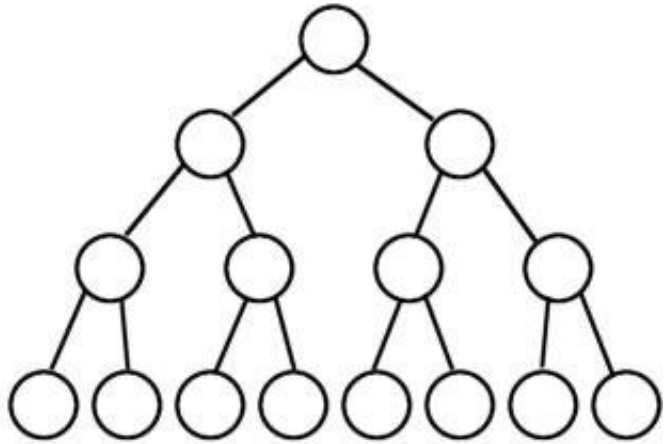
Which of these diagrams would not be optimal for implementing a heap?

Recall ...

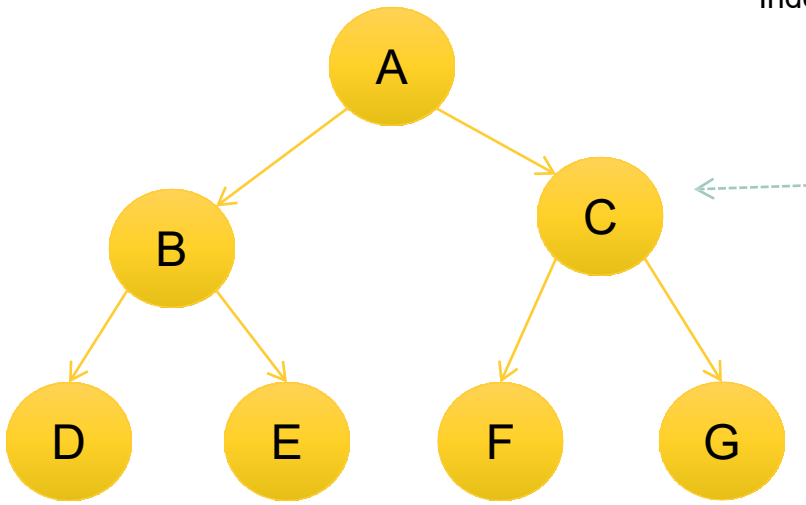
- The depth of a node is the number of edges from the root to the node.
- The height of a node is the number of edges from the node to the deepest leaf.
- The height of a tree is the height of the root.
- A Complete binary tree is a binary tree, which is completely filled, with the possible exception of the bottom level, which is filled from left to right.
- A Perfect binary tree of height **h** is a Complete binary tree that has $2^{h+1}-1$ nodes.
- The height h of a complete binary tree with N nodes is at most O(log N).

h = 3
$2^{(h+1)}-1 = 15$

Full Binary Tree



Representing Binary Tree as an Array



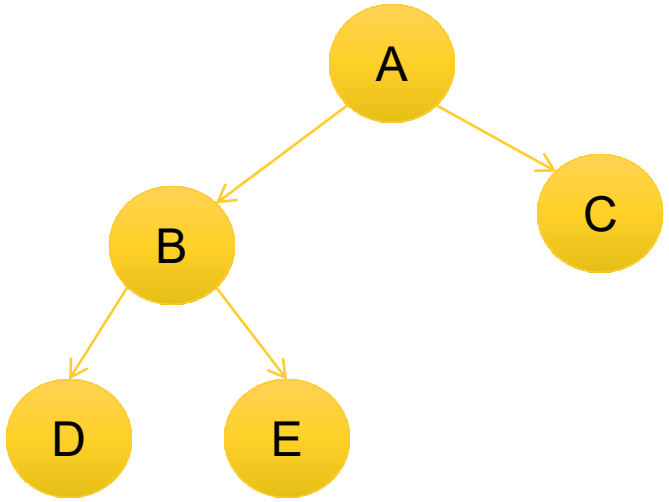
Perfect binary tree

Index:	0	1	2	3	4	5	6	7
		A	B	C	D	E	F	G

Level order traversal

Note:
If a Node is at index i
Then left child is index (2*i)
right child is at index (2*i)+1
parent is at index floor(i/2)

Representing Binary Tree as an Array

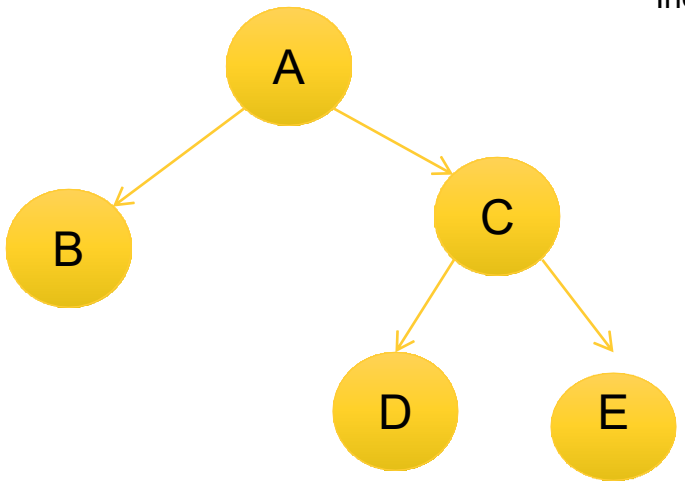


Complete binary tree

Index:	0	1	2	3	4	5	6	7
		A	B	C	D	E		

Note:
If a Node is at index i
Then left child is index (2*i)
right child is at index (2*i)+1
parent is at index floor(i/2)

Representing Binary Tree as an Array

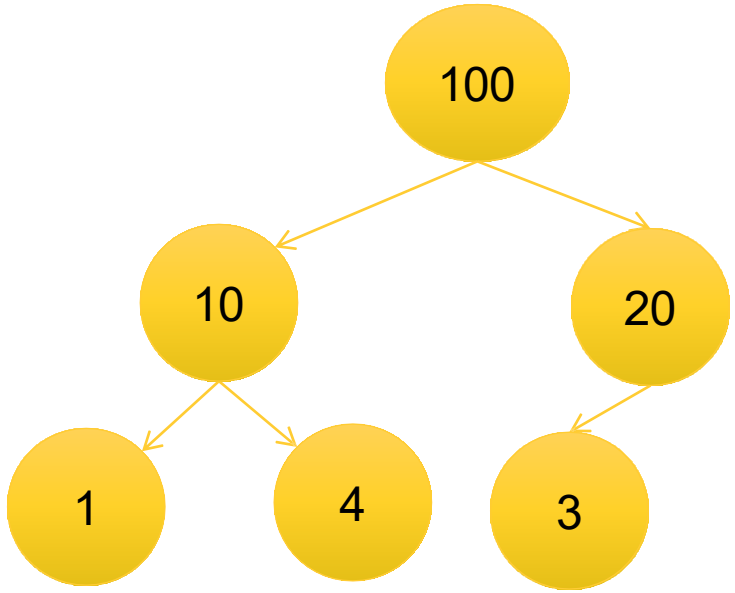


Incomplete binary tree

Index:	0	1	2	3	4	5	6	7
		A	B	C	-	-	D	E

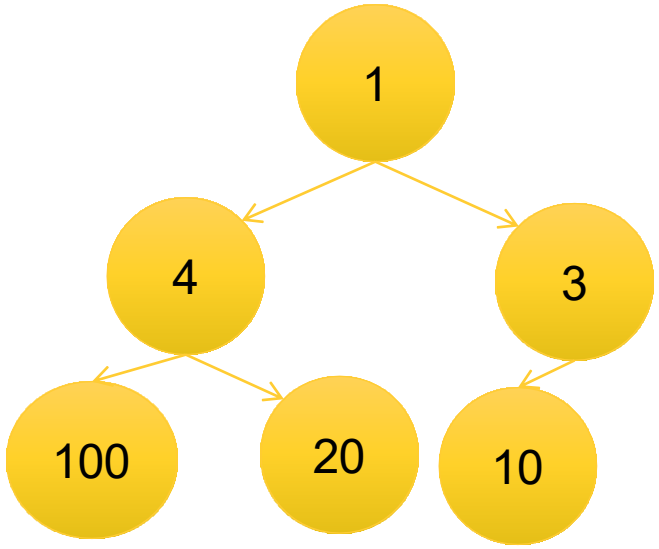
Note:
If a Node is at index i
Then left child is index (2*i)
right child is at index (2*i)+1
parent is at index floor(i/2)

Max/Min Heap



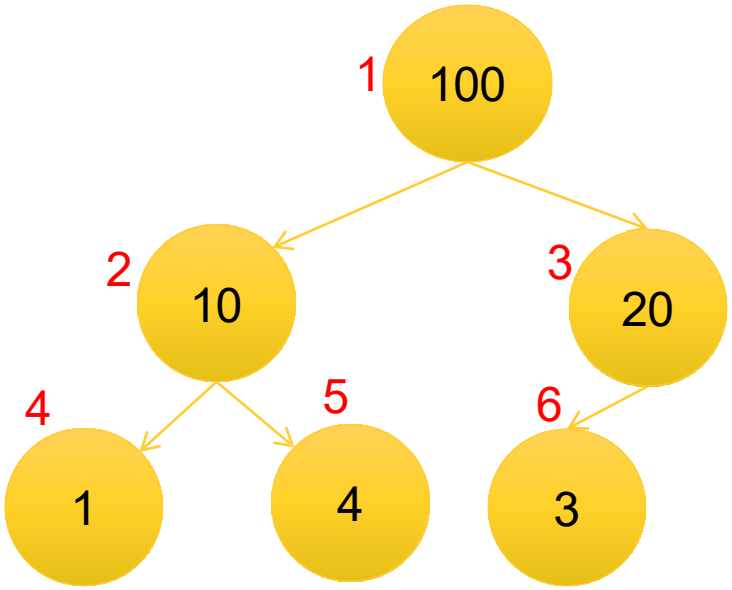
We can visualize a binary heap using a **complete binary tree**

Each node in a Max Heap has a key which is less than (or equal, depending on the application) to the key of its parent.



Each node in a Min Heap has a key which is Greater than (or equal, depending on the application) to the key of its parent.

Max/Min Heap Implementation



0	1	2	3	4	5	6
	100	10	20	1	4	3

Next, we build the tree from the array. For node at I: left child will be 2i while right child will Be (2i + 1). The parent of node I is i/2.

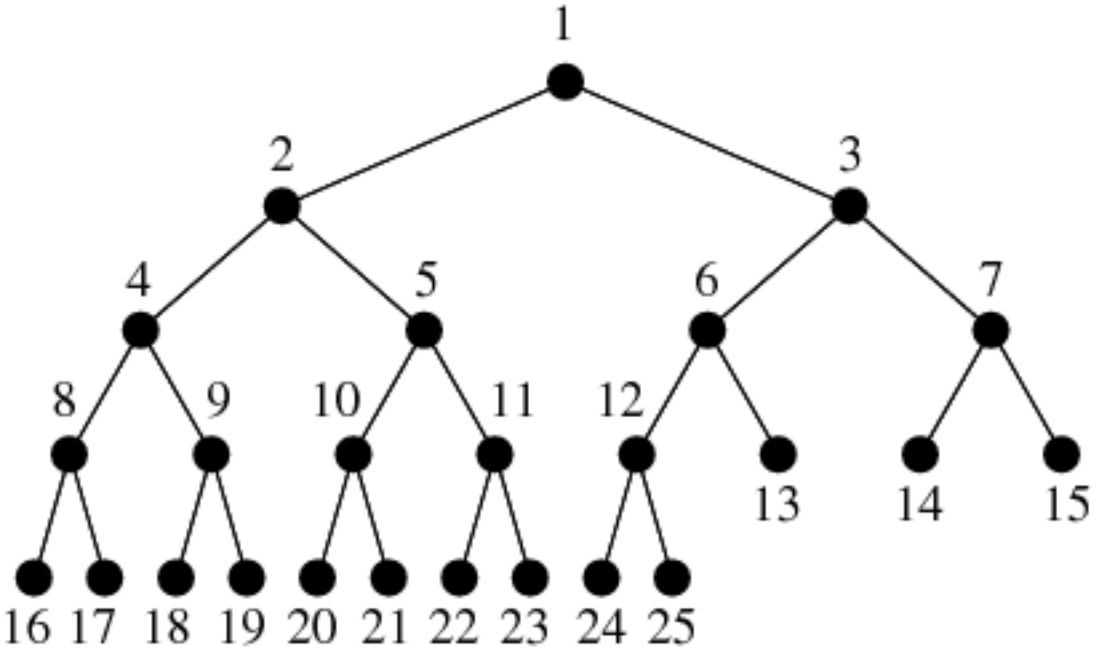
What would result if used an incomplete binary tree?

0	1	2	3	4	5	6	7
	25	12	16	13	10	8	14

Is this a max heap?

Heap

For a complete binary tree with n nodes, leaves occur from ((n/2) + 1) to n.



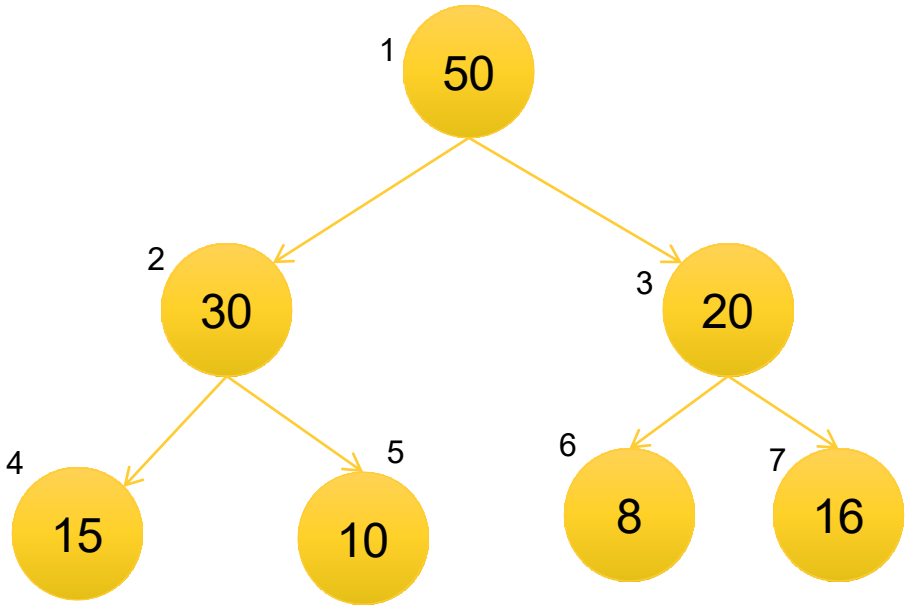
http://mathworld.wolfram.com/CompleteBinaryTree.html

Insertion into a Max Heap

0	1	2	3	4	5	6	7
	50	30	20	15	10	8	16

Let’s say we wish to insert 60?

0	1	2	3	4	5	6	7	8
	50	30	20	15	10	8	16	60



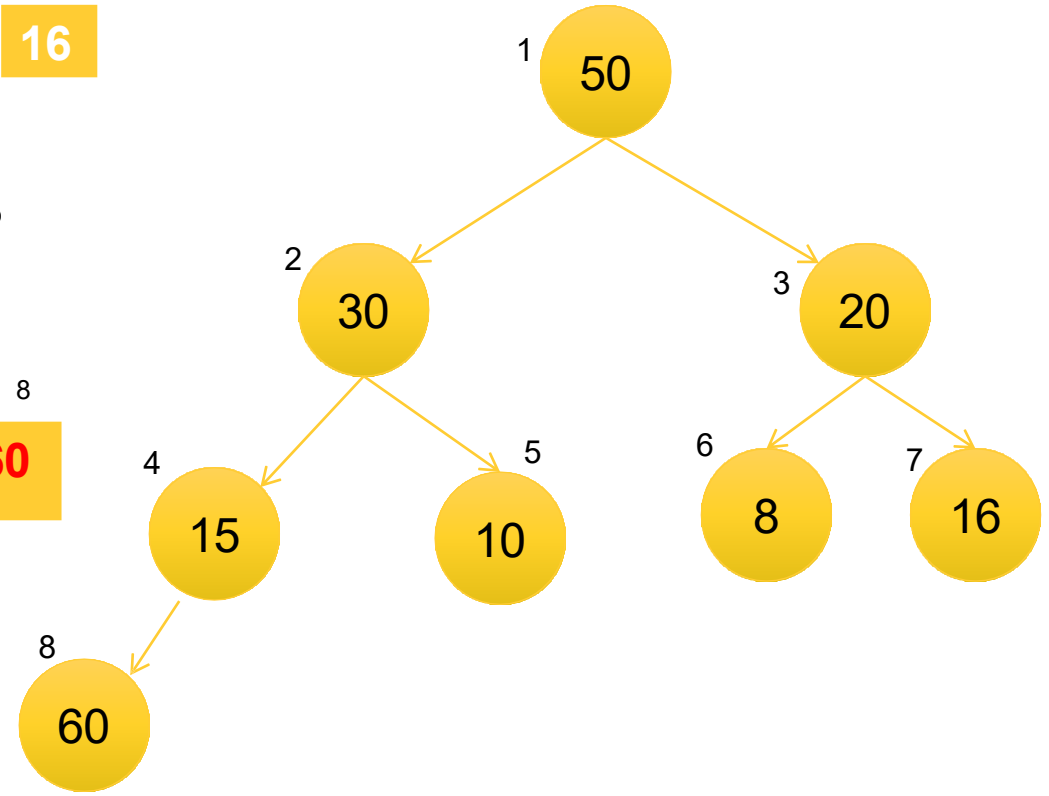
Insertion into a Max Heap

0	1	2	3	4	5	6	7
	50	30	20	15	10	8	16

Let's say we wish to insert 60?

0	1	2	3	4	5	6	7	8
	50	30	20	15	10	8	16	60

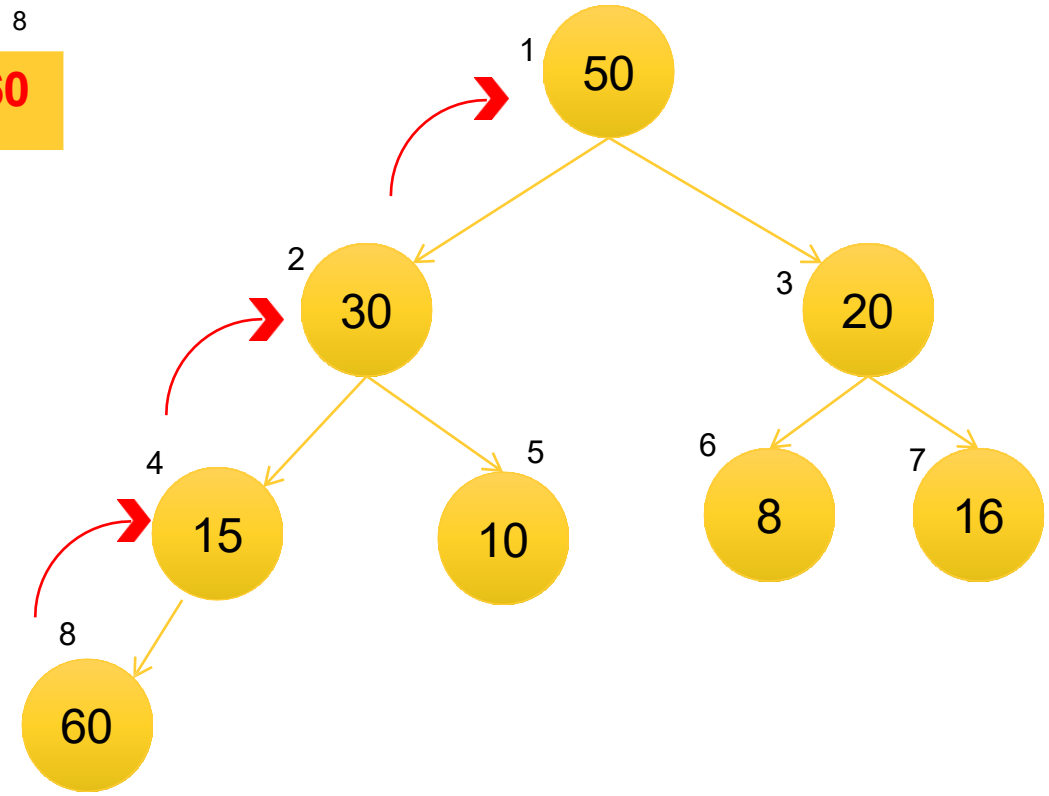
Add the new element as a leaf and adjust upwards (recursively or iteratively) .



Still a Max heap? No.

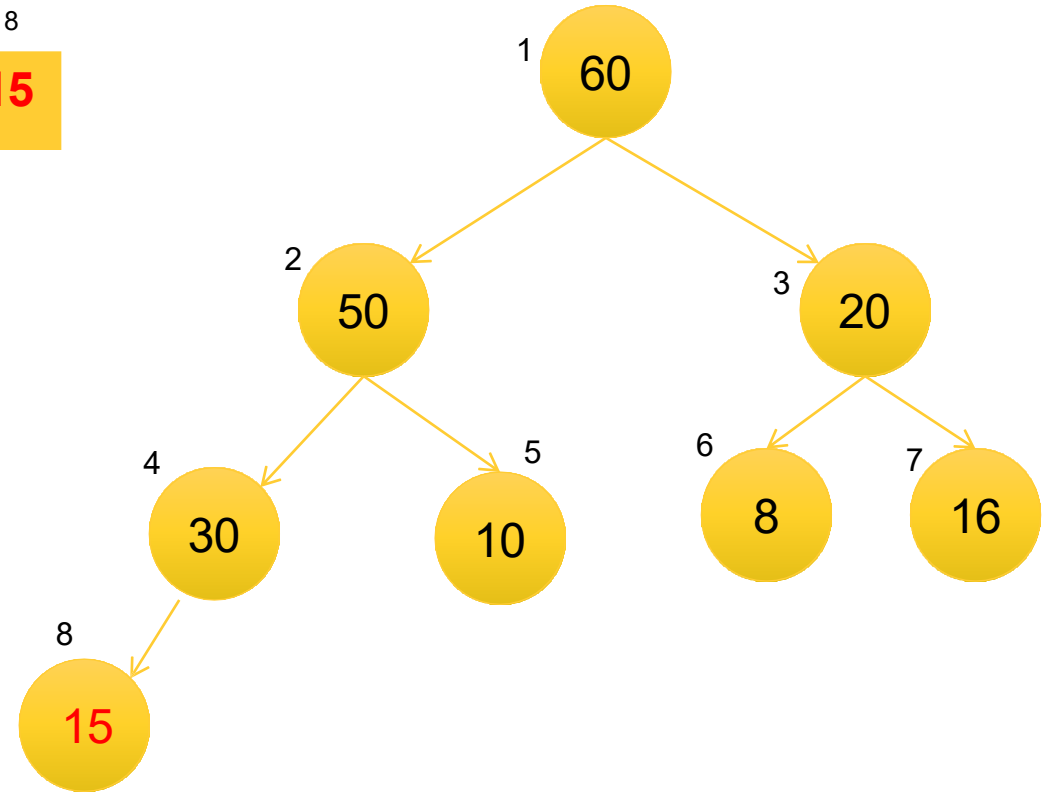
Insertion into a Max Heap

0	1	2	3	4	5	6	7	8
	50	30	20	15	10	8	16	60



Insertion into a Max Heap

0	1	2	3	4	5	6	7	8
	60	50	20	30	10	8	16	15



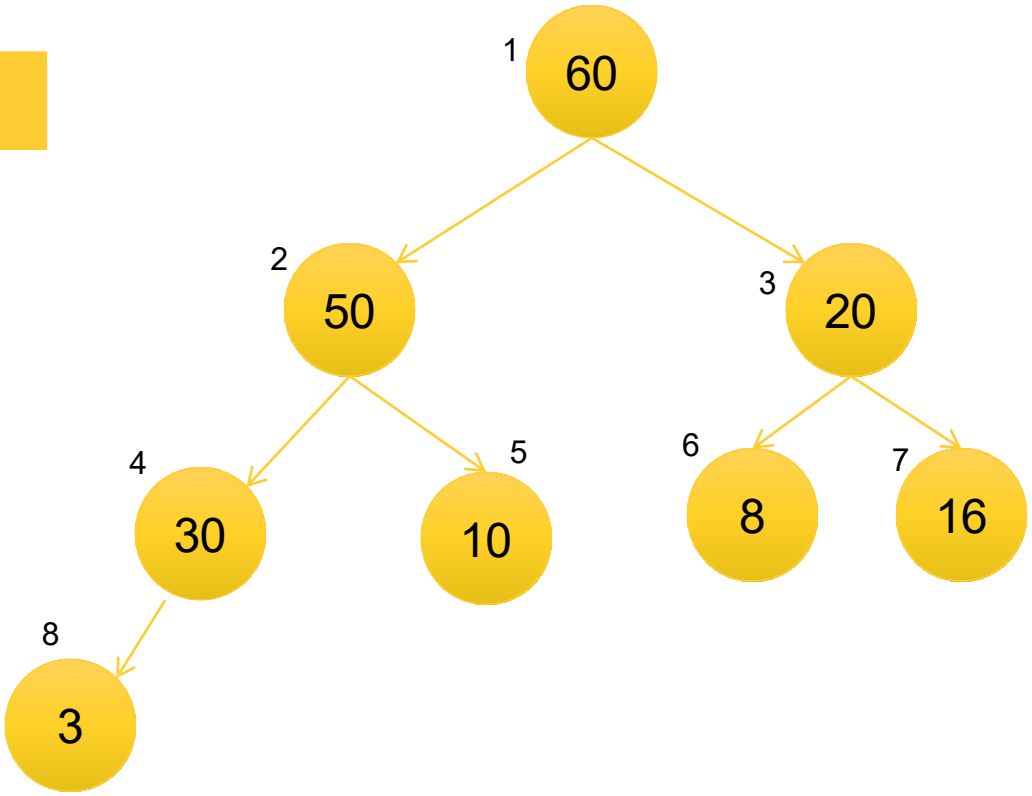
Max heap? Yes.

Time required for this insertion? Function of height, therefore **O(log n)**
What is the time required to build a heap? O(**nlog n**)

Insertion into a Max Heap

0	1	2	3	4	5	6	7	8
	60	50	20	30	10	8	16	3

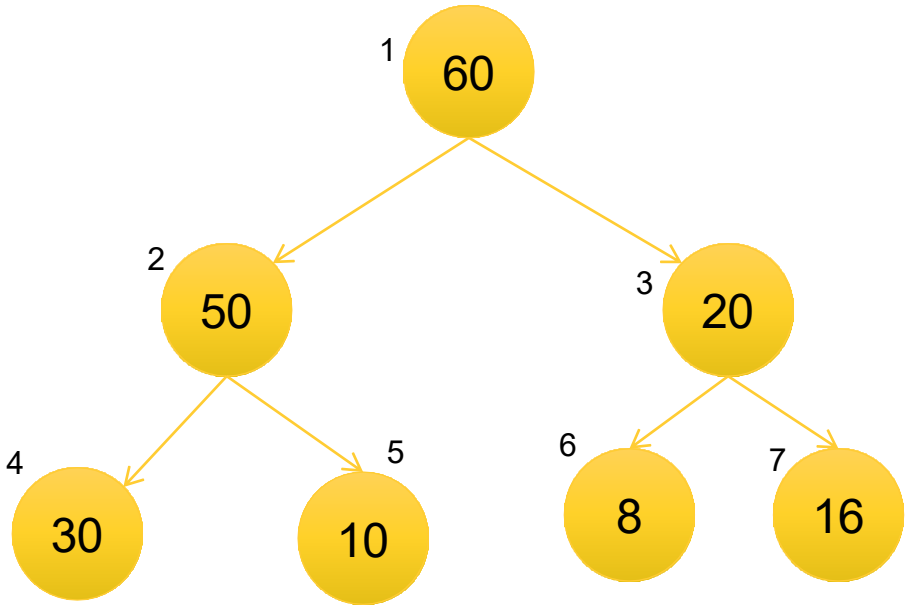
What if we wished to insert 3?
No swaps are necessary, therefore time is O(1)
General time complexity for insertion? **O(1) to O(log n)**



Deletion From a Max Heap

0	1	2	3	4	5	6	7
	60	50	20	30	10	8	16

We only delete the root!
As with insertion, we must preserve the complete binary tree properties.

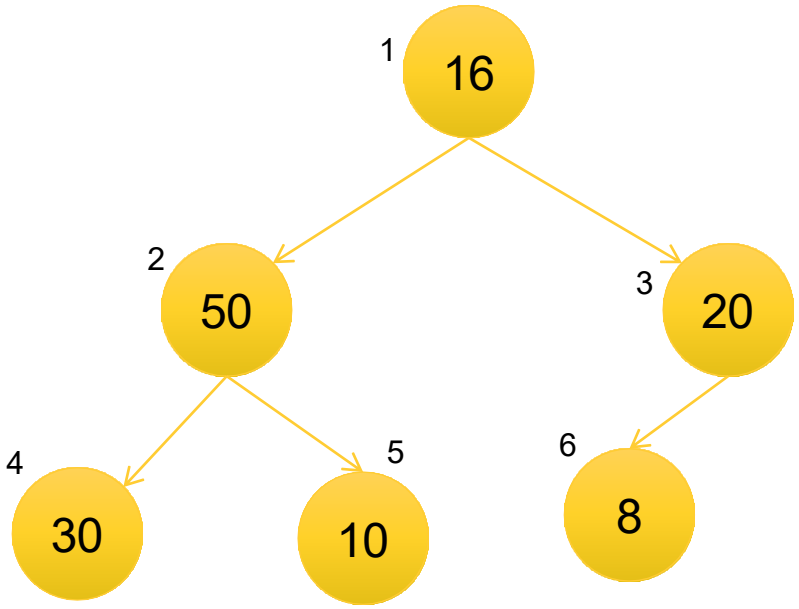


Deletion From a Max Heap

60 is no longer part of the heap, however will keep it in the array.

0	1	2	3	4	5	6		
	16	50	20	30	10	8	60	

Swap the root with the last element in the heap.

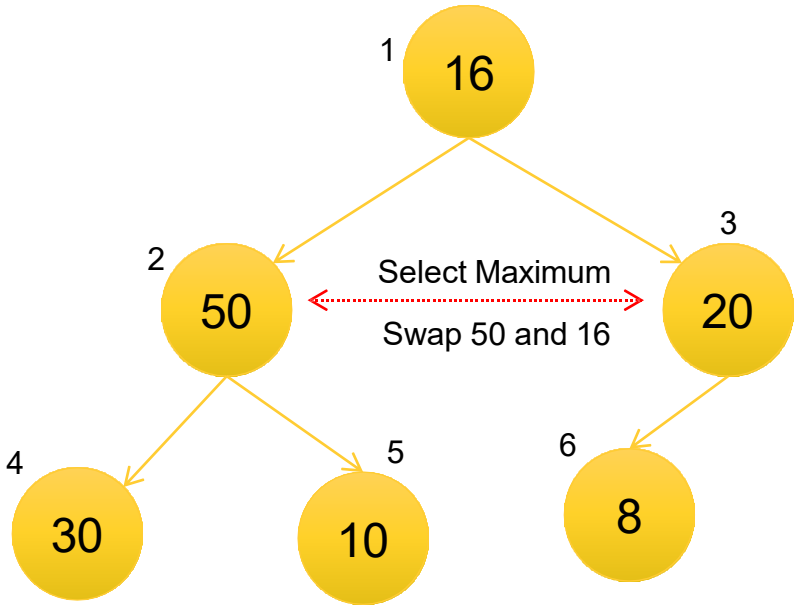


Still a Max heap? No.

Deletion From a Max Heap

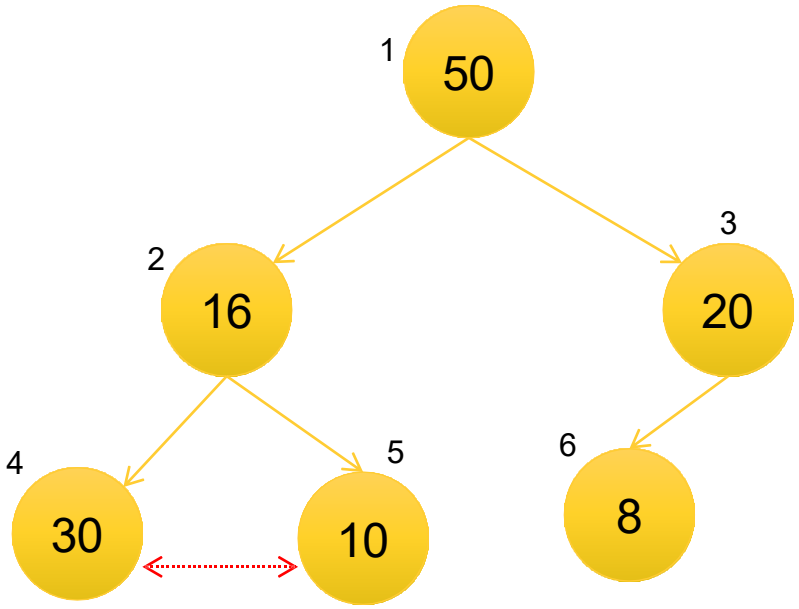
0	1	2	3	4	5	6		
	16	**50**	20	30	10	8	**60**	

Now we recursively work our way down the tree.



Deletion From a Max Heap

0	1	2	3	4	5	6		
	50	16	20	30	10	8	60	

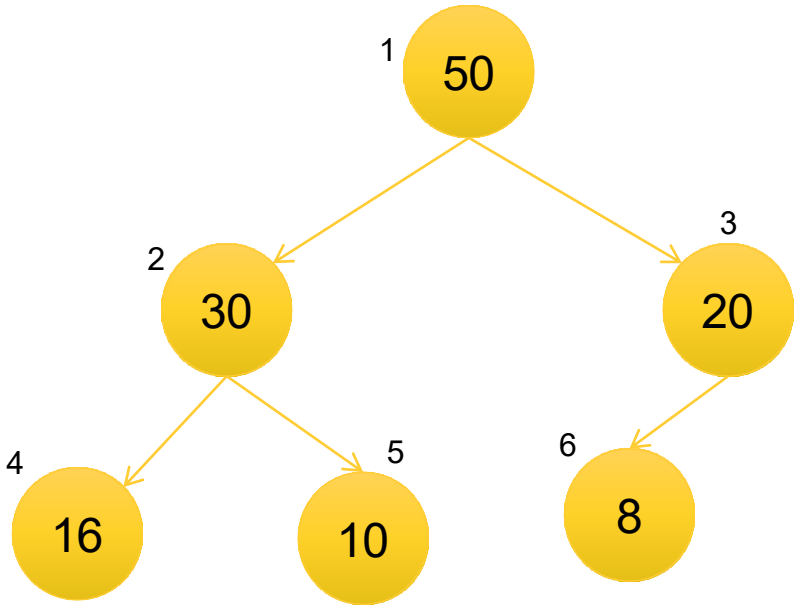


Select Maximum
Swap 16 and 30

Continue to recursively work down the tree until Max Heap properties are achieved.

Deletion From a Max Heap

0	1	2	3	4	5	6		
	50	30	20	16	10	8	**60**	



Insertion: Adjust up the tree.
Deletion: Adjust down the tree.

Max Heap: Deletion produces largest element
Min Heap: Deletion produces smallest element

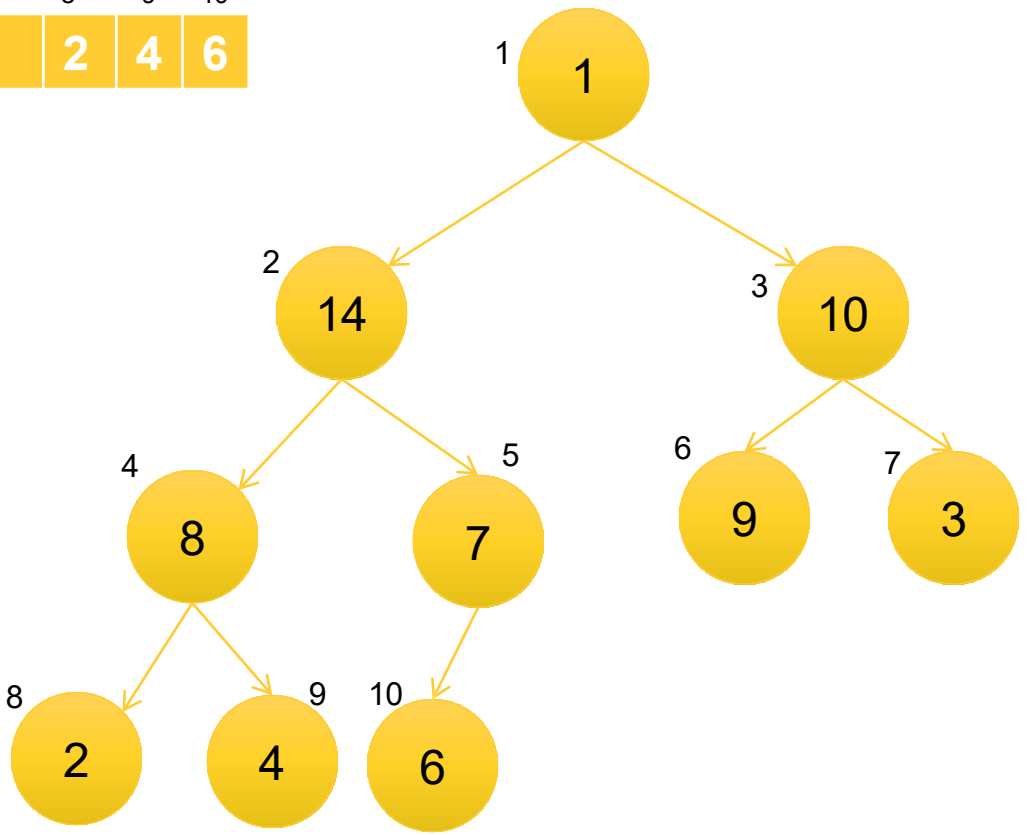
Time required for individual element deletion? Function of height, therefore **O(log n)**
How much time does it take to delete all elements from the heap? **O(nlog n)**

Build a Max Heap from an unsorted Array (*Heapify*)

Start with unsorted array A[]:

0	1	2	3	4	5	6	7	8	9	10
	1	14	10	8	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else
        largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```



Correct a single violation of the heap property in a subtree's root – recursively.

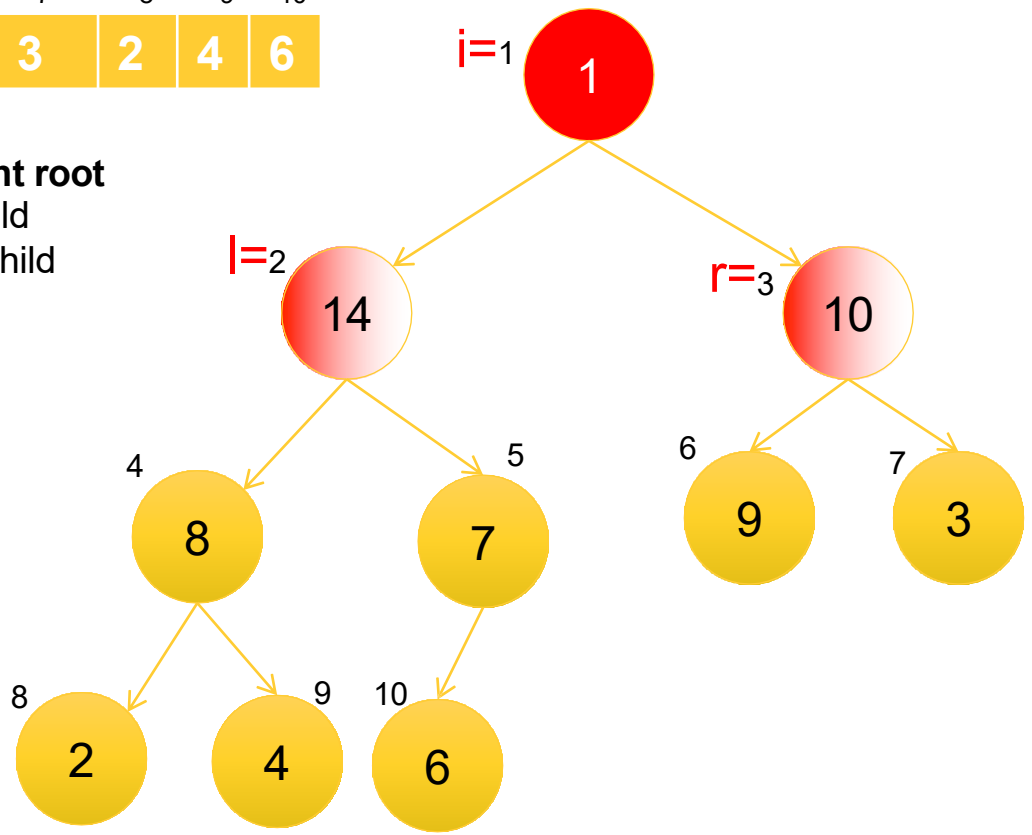
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	1	14	10	8	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



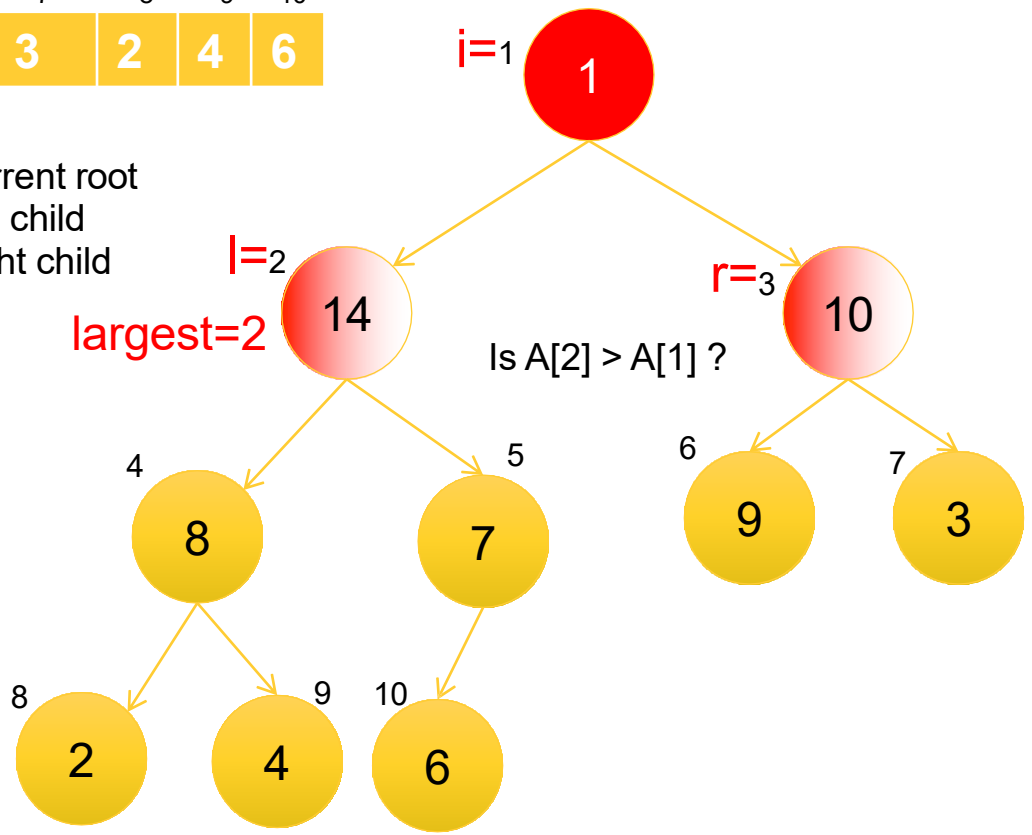
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	1	14	10	8	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



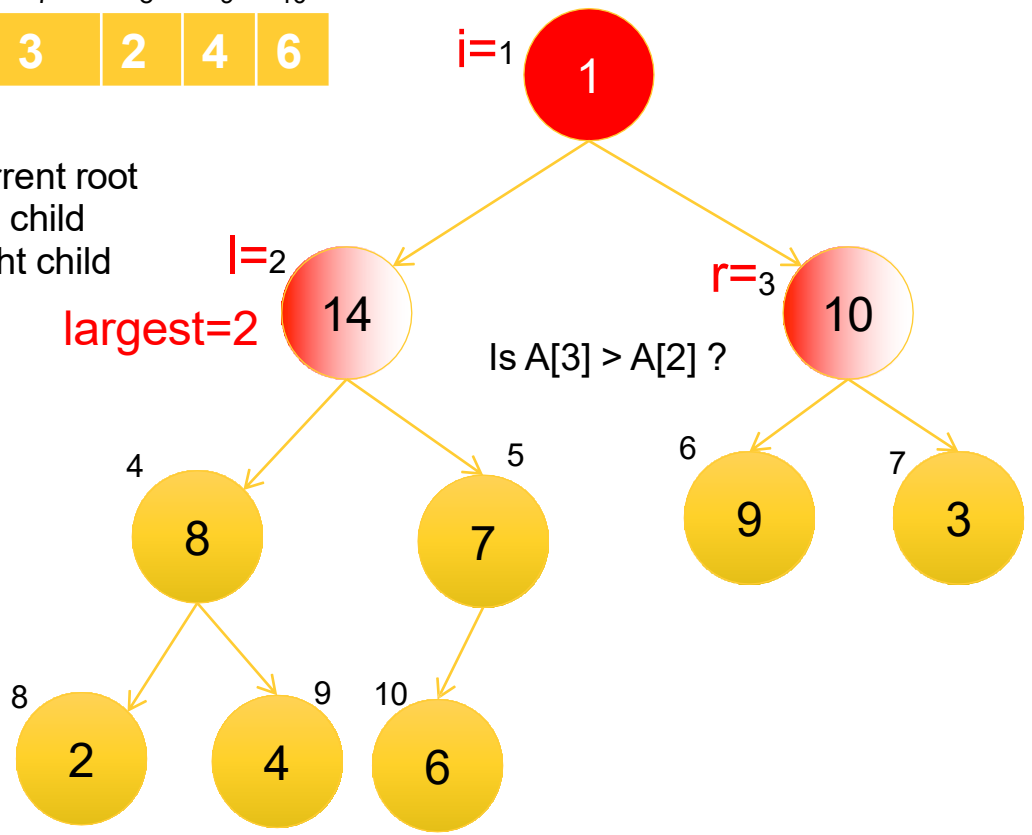
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	1	14	10	8	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



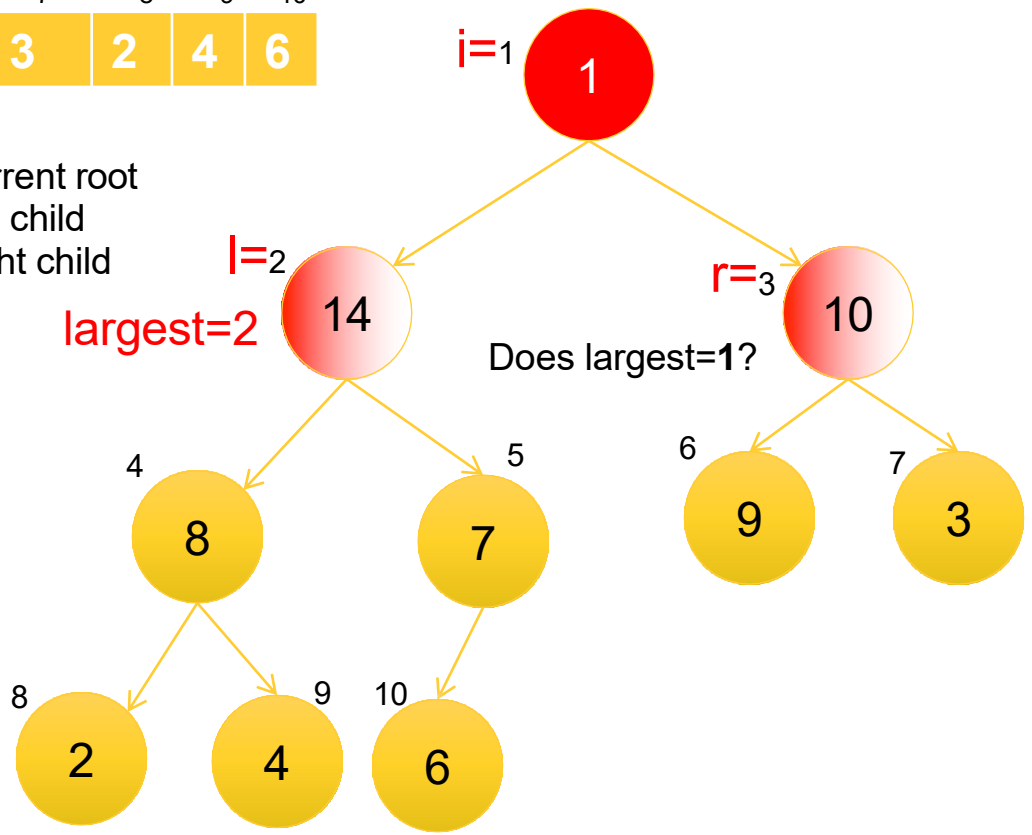
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	1	14	10	8	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



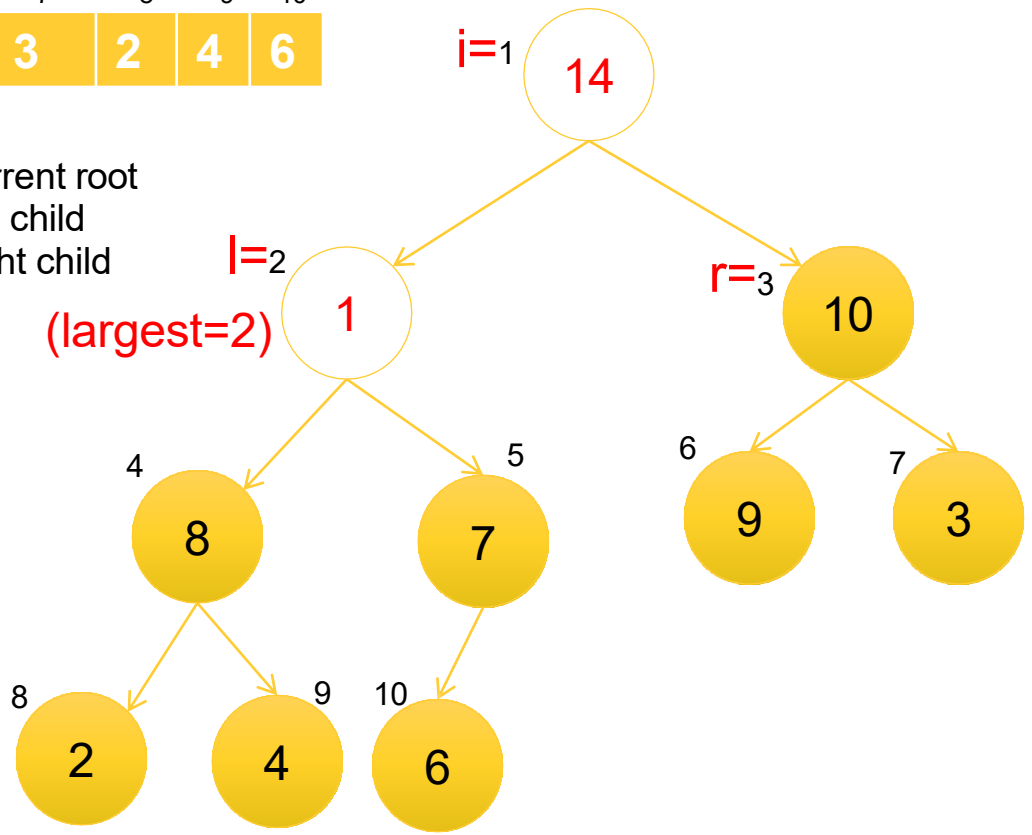
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	1	10	8	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



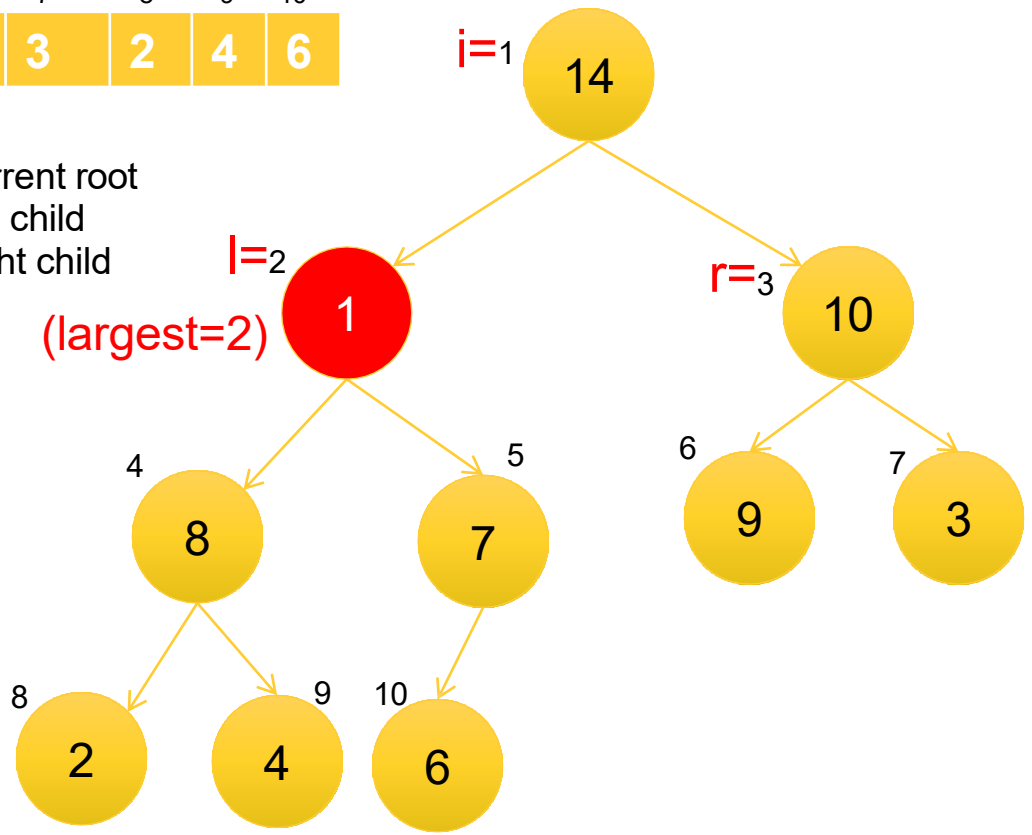
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	1	10	8	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



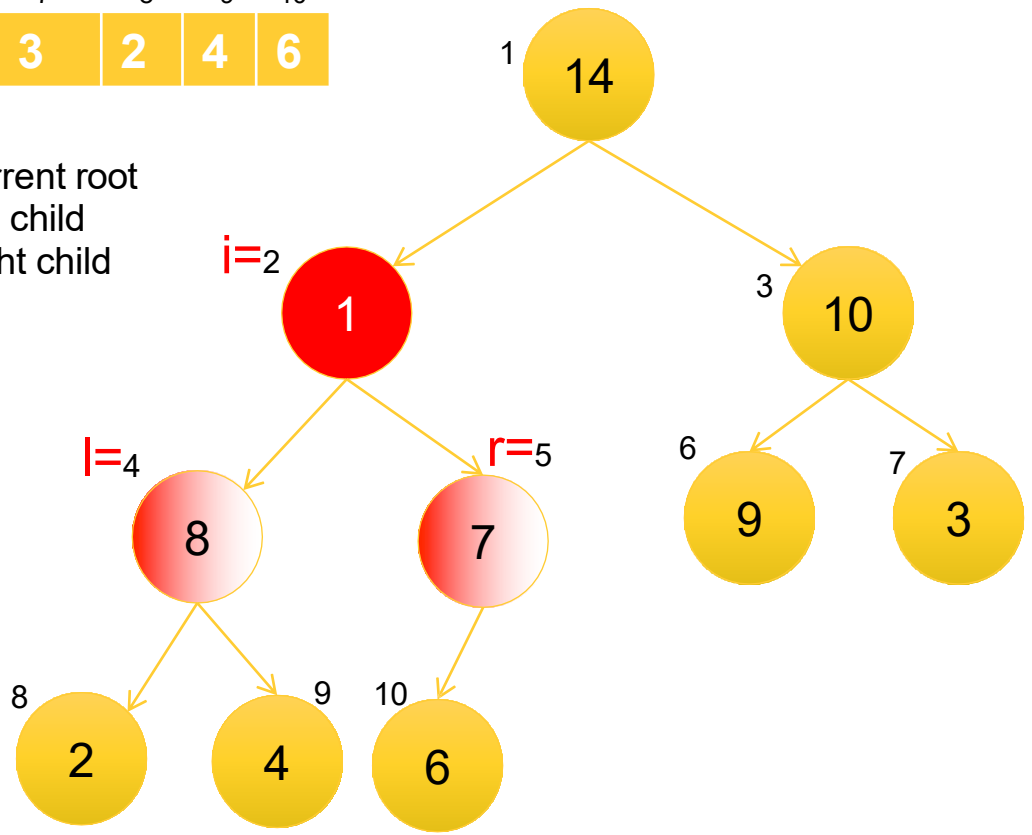
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	1	10	8	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



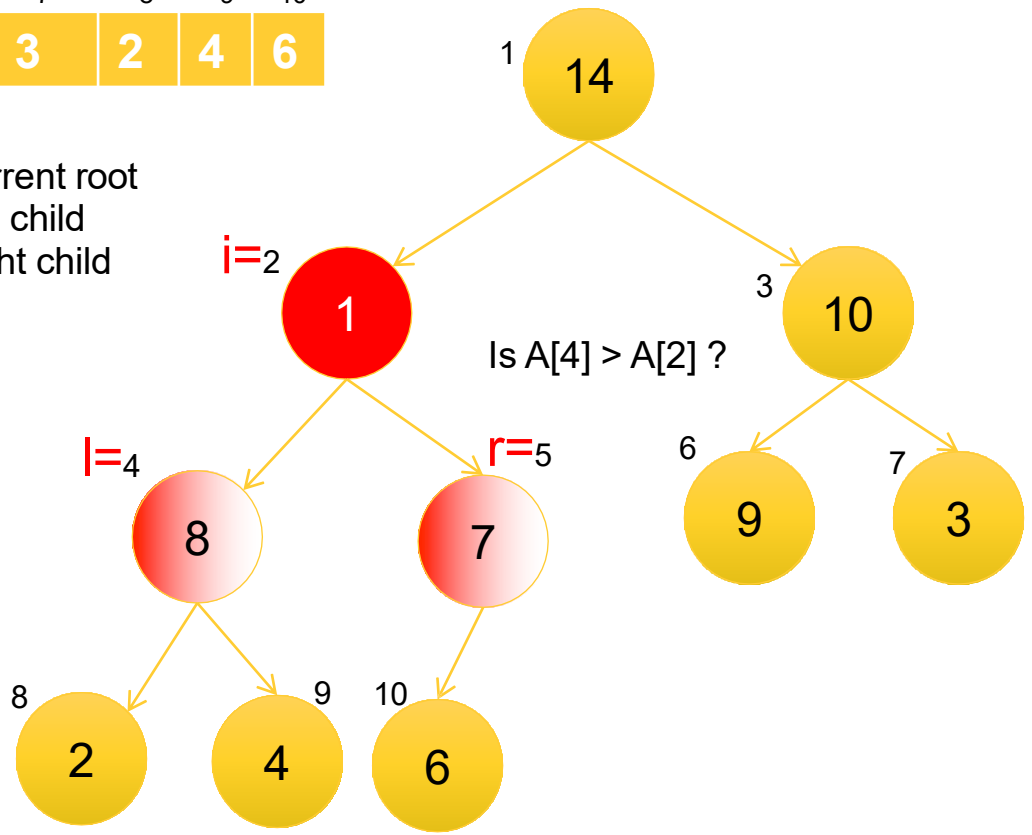
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	1	10	8	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



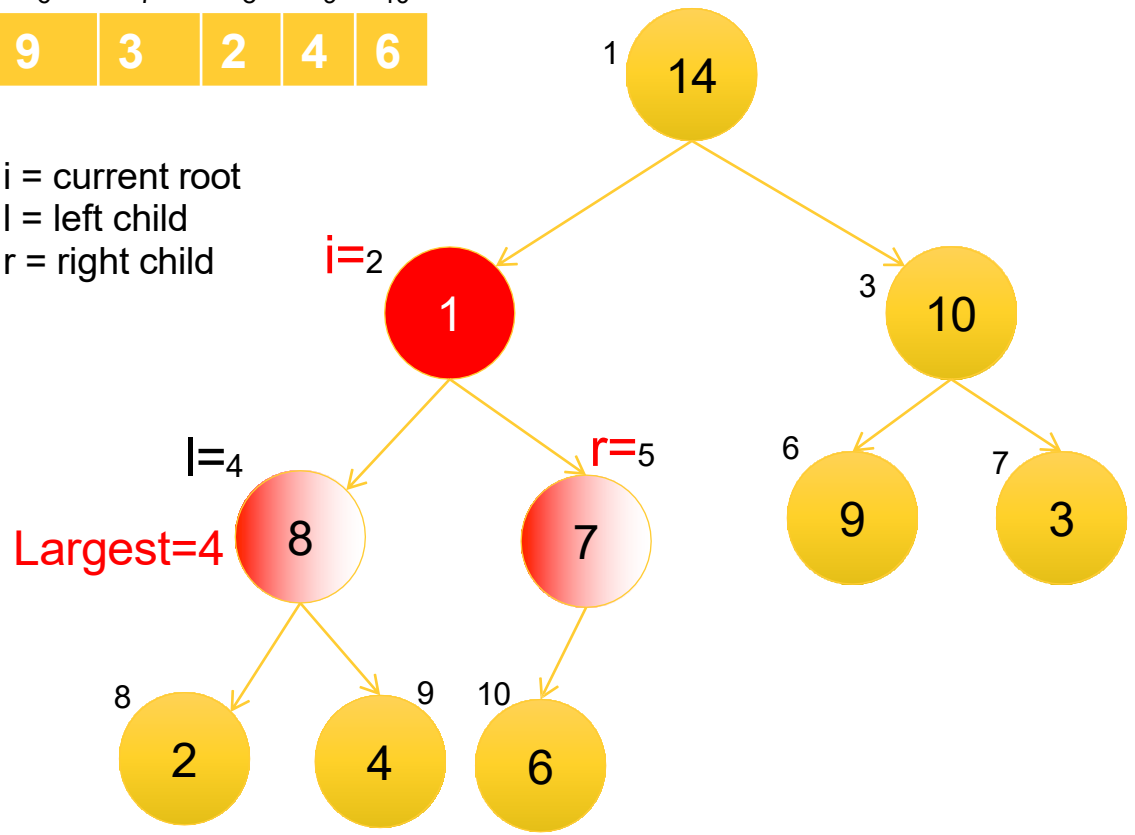
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	1	10	8	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



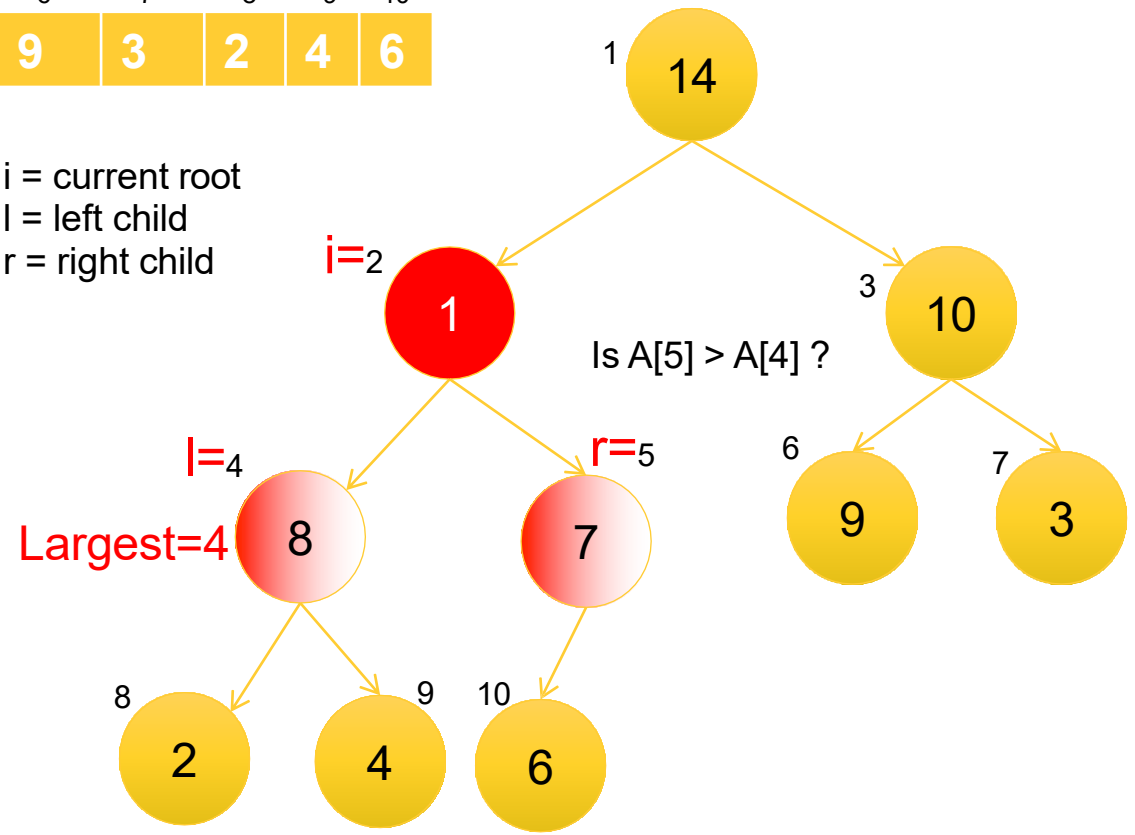
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	1	10	8	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



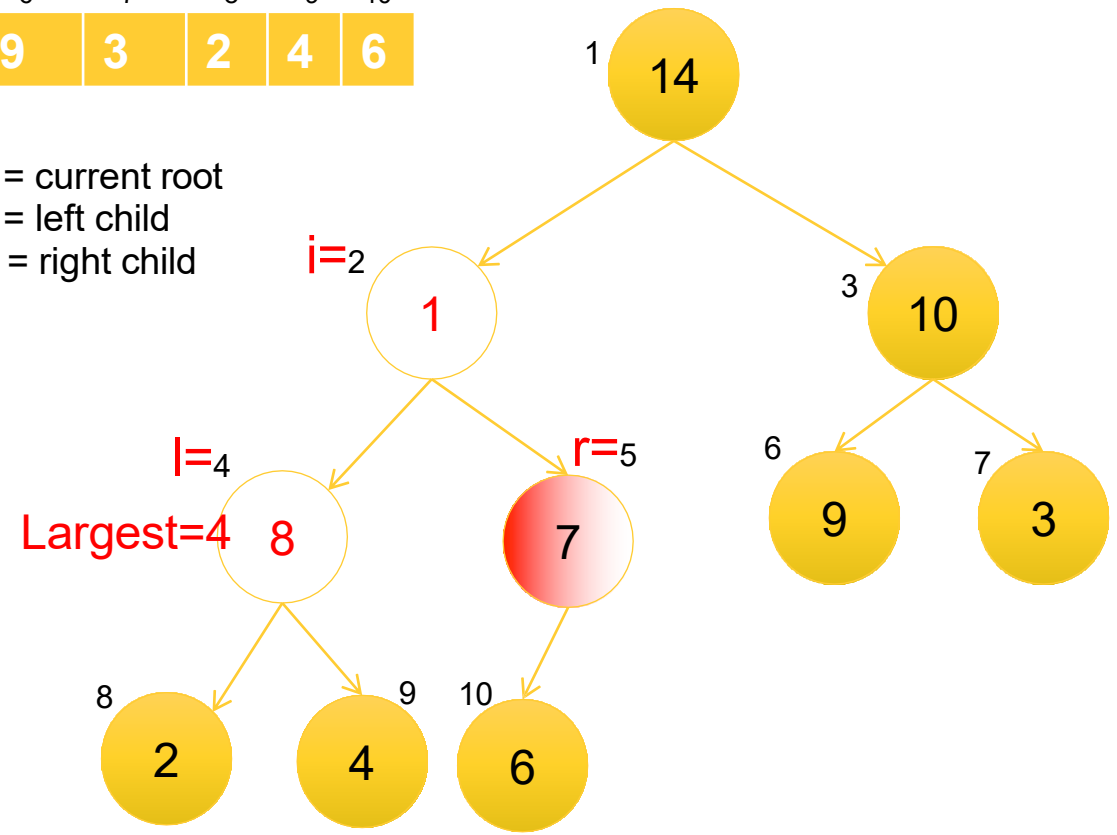
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	8	10	1	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



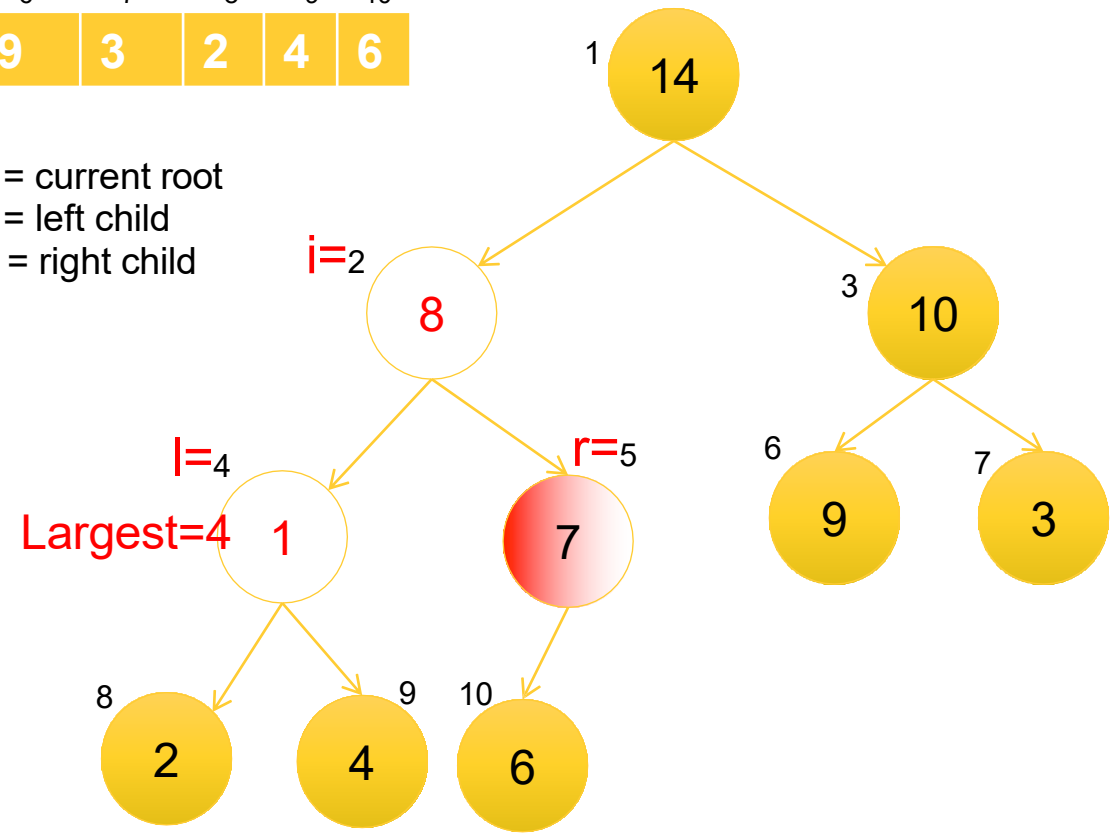
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	8	10	1	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



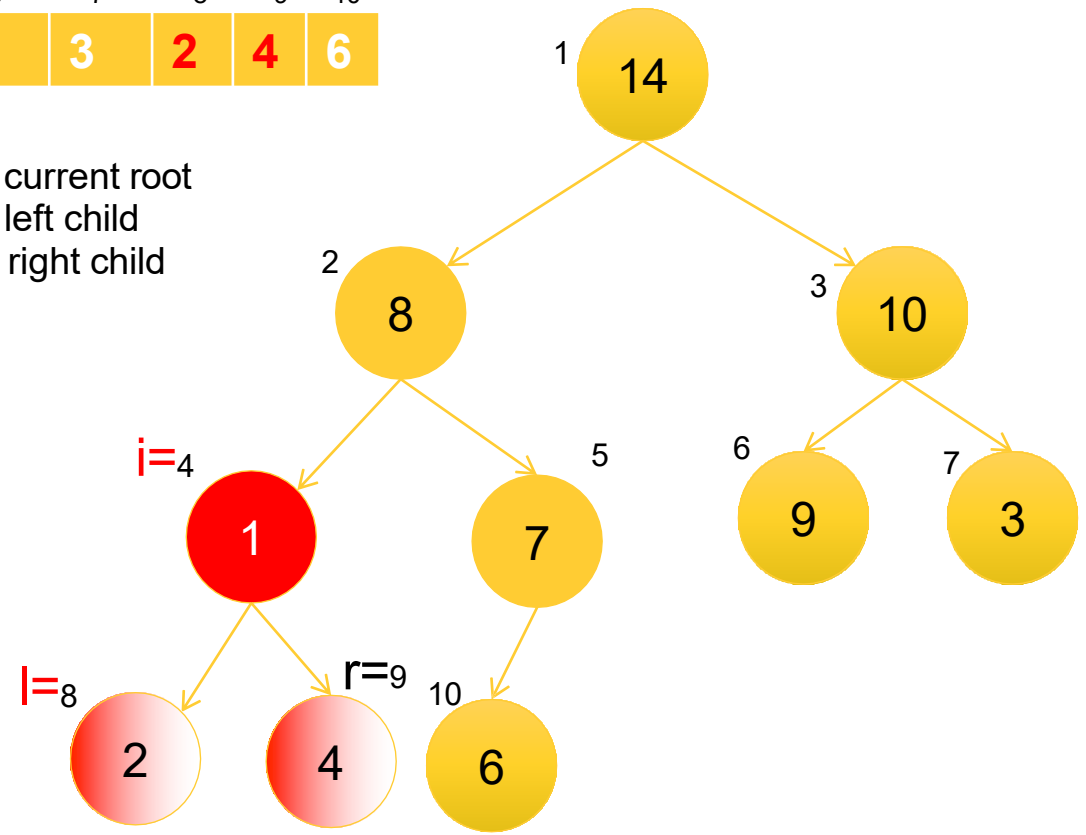
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	8	10	1	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = i;
    else  largest = l;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



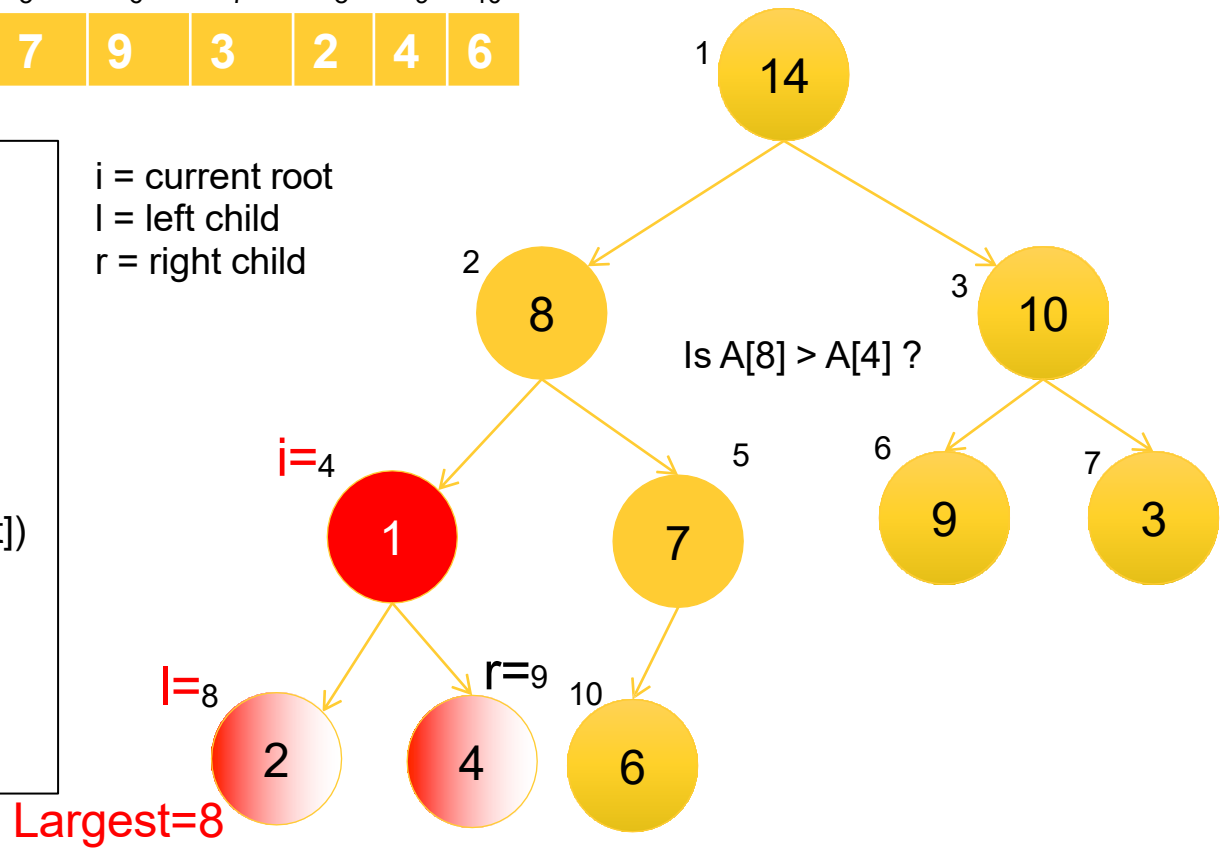
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	8	10	1	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



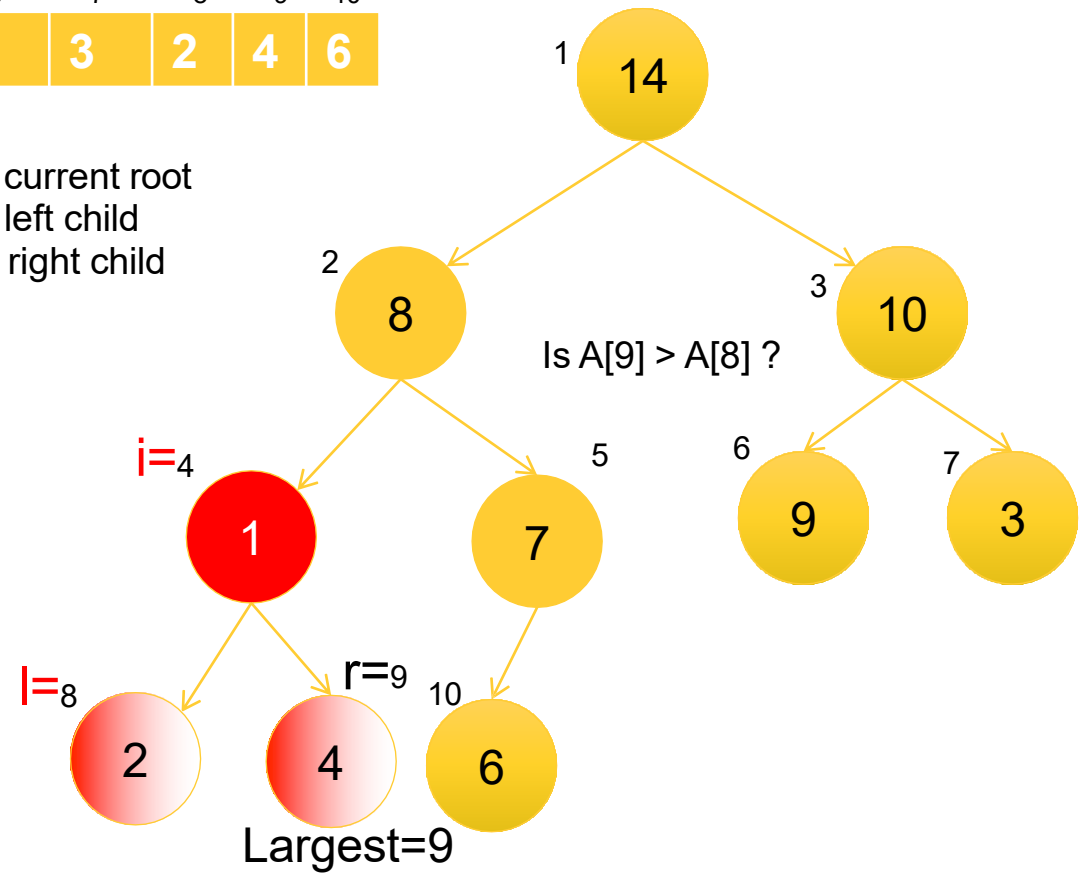
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	8	10	1	7	9	3	2	4	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



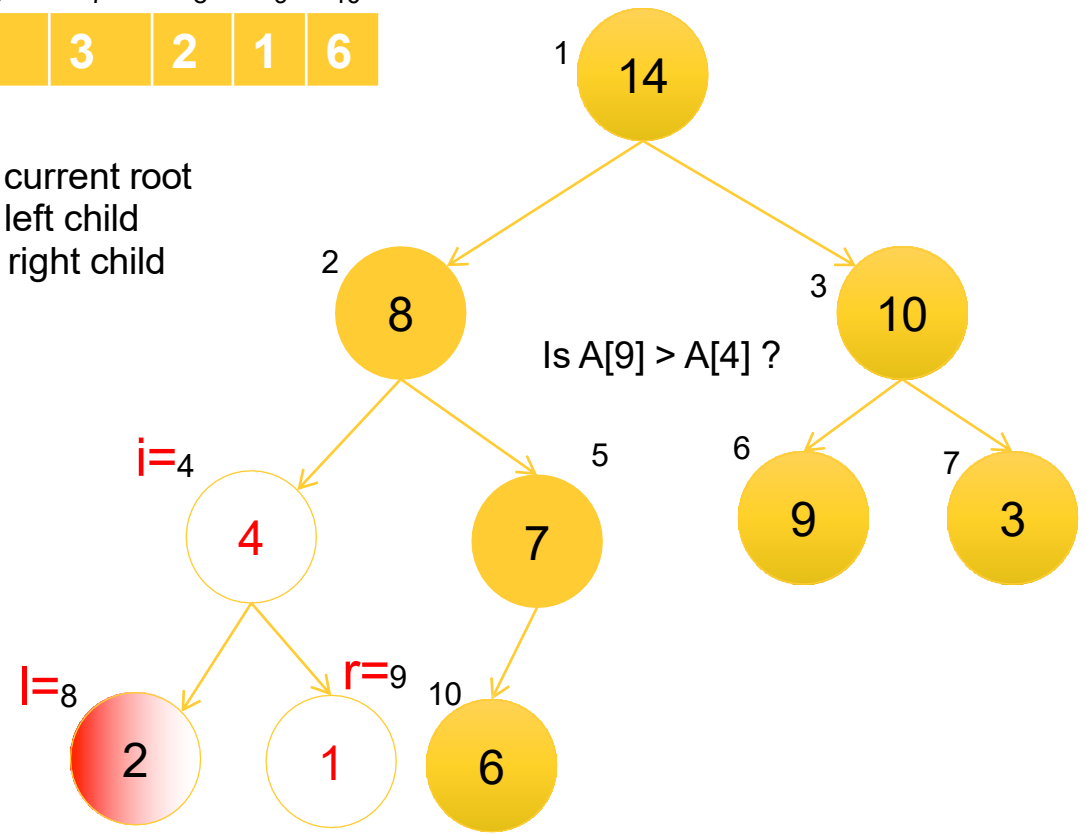
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	8	10	4	7	9	3	2	1	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



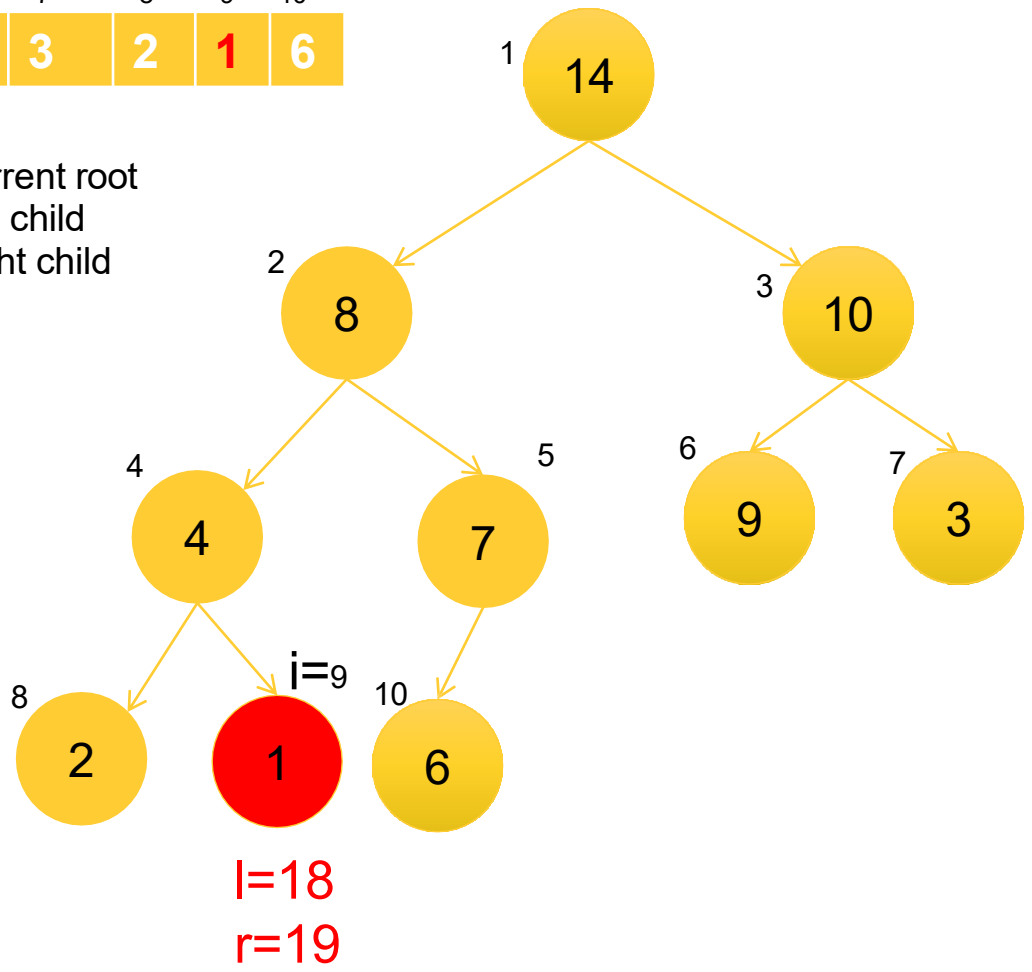
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	8	10	4	7	9	3	2	1	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



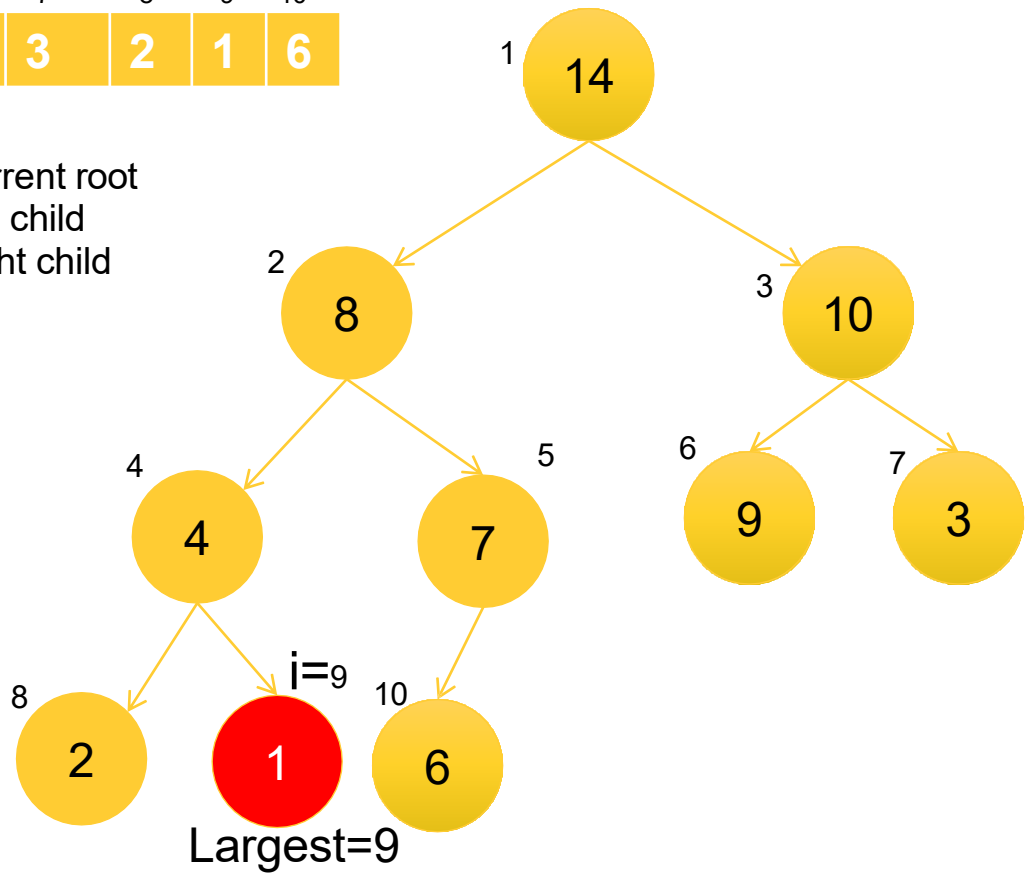
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	8	10	4	7	9	3	2	1	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i; //largest remains i
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



l=18 is 18 <= 10?
r=19

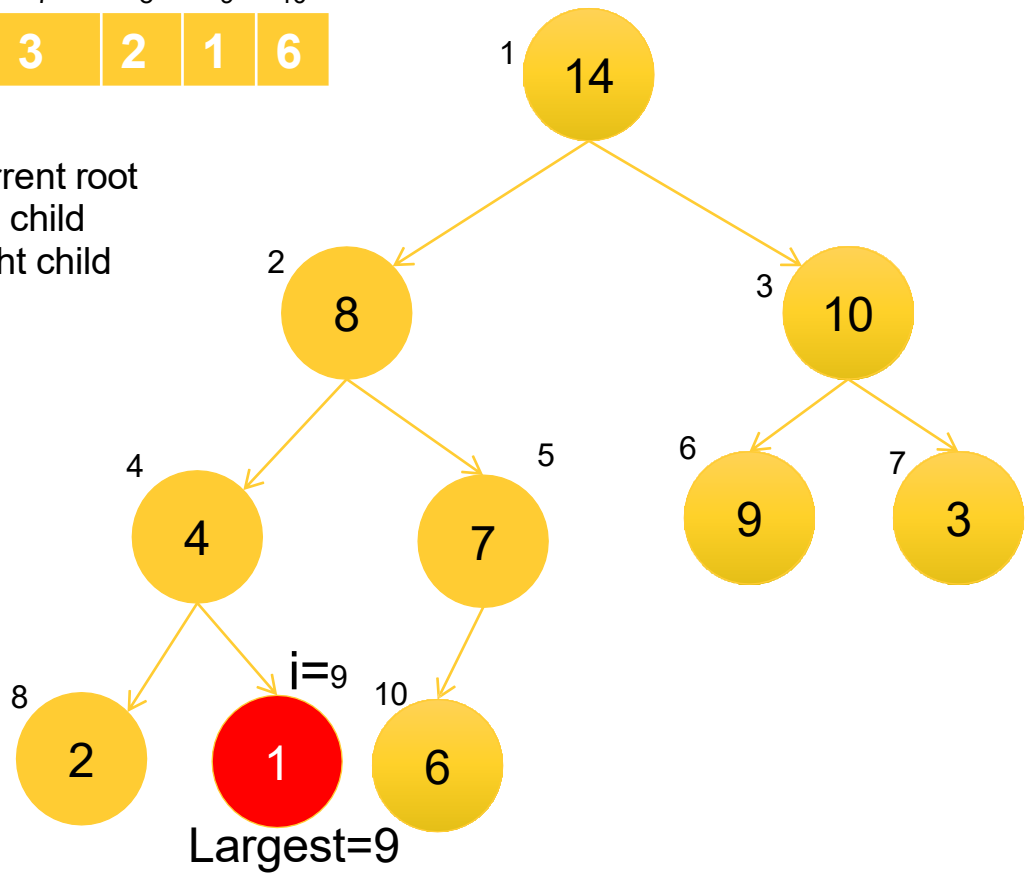
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	8	10	4	7	9	3	2	1	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child



l=18
r=19 is 19 <= 10?

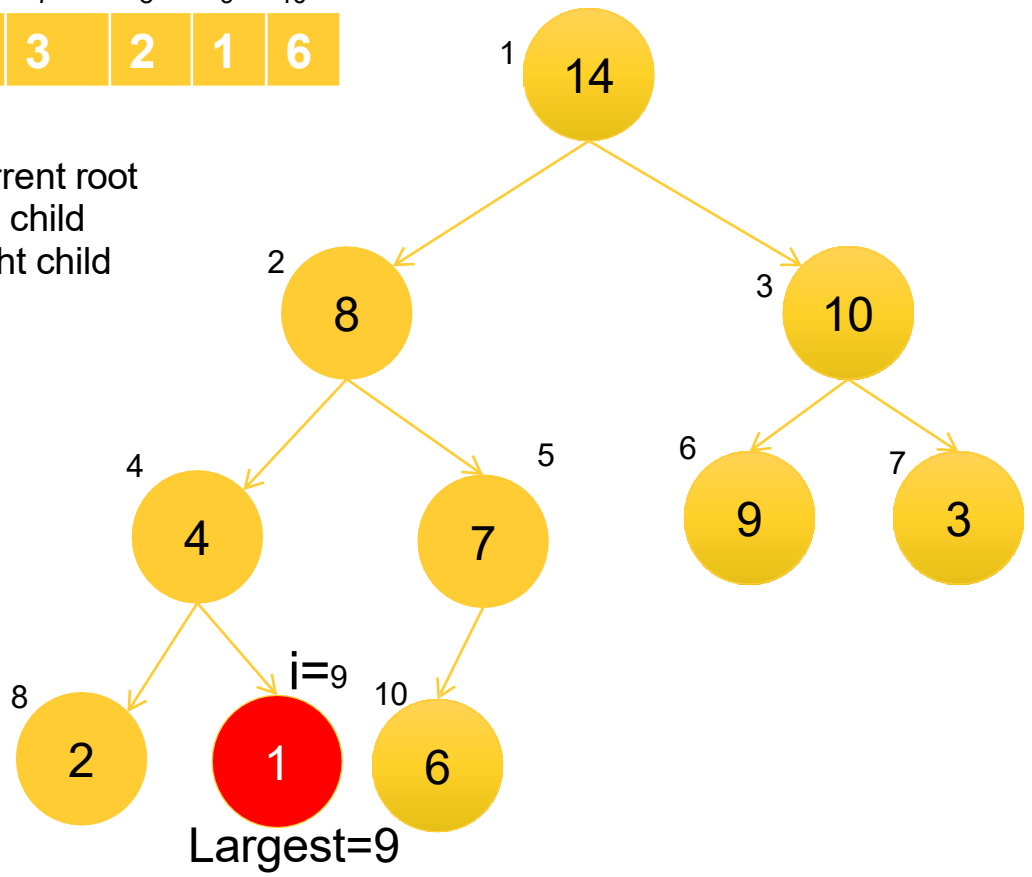
Build a Max Heap from an unsorted Array (*Heapify*)

Start with array A[]:

0	1	2	3	4	5	6	7	8	9	10
	14	8	10	4	7	9	3	2	1	6

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

i = current root
l = left child
r = right child

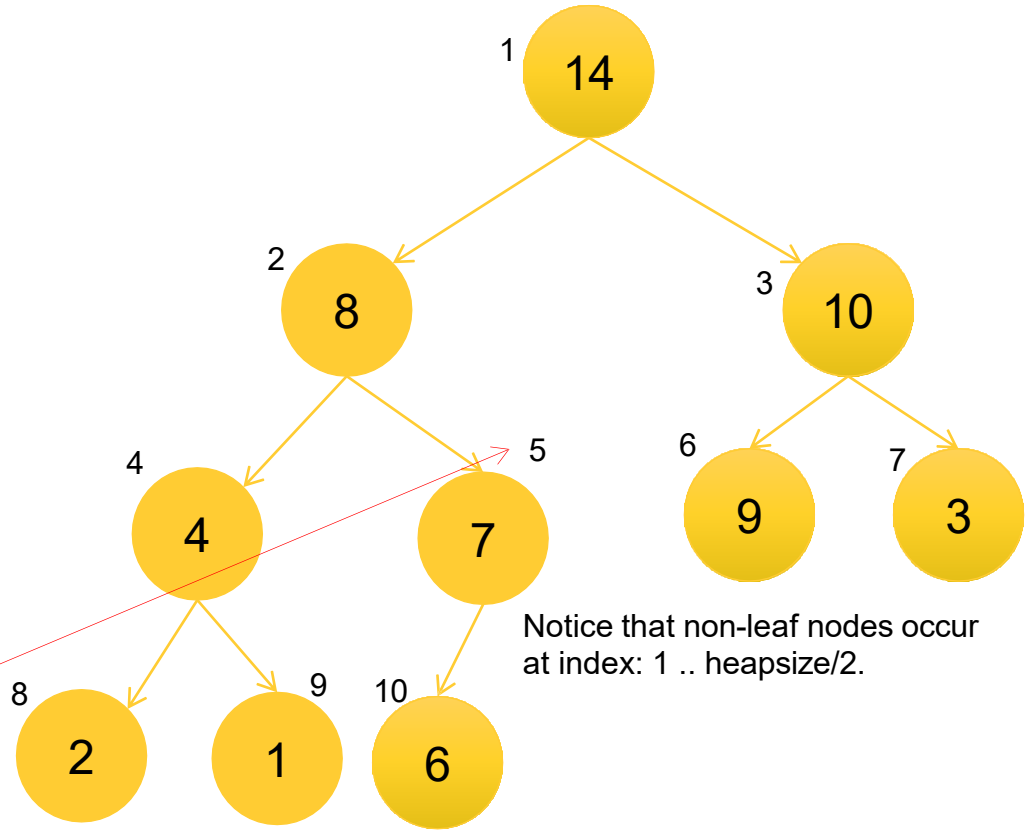


What is the complexity of maxHeapify?

Build a Max Heap from an unsorted Array (*Heapify*)

```
//pseudocode
maxHeapify(A[],i)
{
    l = 2i;
    r= 2i +1;
    if (l <= A.heapSize and A[l]>A[i])
        largest = l;
    else  largest = i;
    if (r <= A.heapSize and A[r]>A[largest])
        largest = r;
    if (largest != i)
        exchange A[i] with A[largest]
        maxHeapify(A,largest)
}
```

```
BuildMaxHeap(A)
{
   A.heapSize = A.length
   for (i =  (A.length/2) down to 1)
       maxHeapify(A,i);
}
```



Notice that non-leaf nodes occur at index: 1 .. heapsize/2.